

# Personal Loan Application System

## API Design Document

Version: 1.0 | Status: In Progress

| Property       | Value                            |
|----------------|----------------------------------|
| Document Type  | API Design Document              |
| Project        | Personal Loan Application System |
| Base URL       | /api/v1                          |
| Protocol       | HTTPS                            |
| Auth           | JWT Bearer Token                 |
| Content Type   | application/json                 |
| Total Sections | 13                               |

## Table of Contents

---

| #  | Section             |
|----|---------------------|
| 1  | API Standards       |
| 2  | Authentication APIs |
| 3  | Customer APIs       |
| 4  | Loan APIs           |
| 5  | Document APIs       |
| 6  | Approval APIs       |
| 7  | EMI APIs            |
| 8  | Notification APIs   |
| 9  | Admin APIs          |
| 10 | Error Responses     |
| 11 | Status Codes        |
| 12 | API Security        |
| 13 | API Versioning      |

# 1. API Standards

---

## Purpose

The API Standards define the design guidelines, conventions, and best practices followed while building REST APIs for the Personal Loan Application System. These standards ensure consistency, security, and scalability across all API endpoints.

## Base URL

```
https://api.personalloan.com/api/v1
```

## General Standards

| Property       | Standard                               |
|----------------|----------------------------------------|
| Protocol       | HTTPS only                             |
| API Style      | RESTful                                |
| Data Format    | JSON (application/json)                |
| Auth Mechanism | JWT Bearer Token                       |
| API Version    | v1 (in URL path)                       |
| Date Format    | ISO 8601 (yyyy-MM-ddTHH:mm:ssZ)        |
| Charset        | UTF-8                                  |
| Pagination     | Page-based (page, size, totalElements) |

## HTTP Method Standards

| HTTP Method | Purpose              | Example                             |
|-------------|----------------------|-------------------------------------|
| GET         | Retrieve data        | GET /api/v1/loans                   |
| POST        | Create new resource  | POST /api/v1/loans                  |
| PUT         | Update full resource | PUT /api/v1/loans/{loanId}          |
| PATCH       | Partial update       | PATCH /api/v1/loans/{loanId}/status |
| DELETE      | Delete resource      | DELETE /api/v1/documents/{docId}    |

## URL Naming Standards

- Use lowercase letters.

- Use hyphens (-) to separate words in URLs.
- Use plural nouns for resource names.
- Avoid verbs in URLs — use HTTP methods instead.
- Use path parameters for specific resources.
- Use query parameters for filtering, sorting, and pagination.

```
// CORRECT
GET /api/v1/loan-applications
GET /api/v1/loan-applications/{loanId}
POST /api/v1/loan-applications

// WRONG
GET /api/v1/getLoanApplications
POST /api/v1/createLoan
```

## Request Header Standards

| Header           | Value                     | Required                |
|------------------|---------------------------|-------------------------|
| Content-Type     | application/json          | Yes                     |
| Authorization    | Bearer <JWT_TOKEN>        | Yes (secured endpoints) |
| Accept           | application/json          | Yes                     |
| X-Request-ID     | UUID (unique per request) | Optional                |
| X-Correlation-ID | Tracing ID                | Optional                |

## Standard Response Wrapper

```
{
  "success" : true,
  "message" : "Operation successful",
  "data" : { },
  "timestamp" : "2026-06-28T10:00:00Z",
  "statusCode": 200
}
```

## Standard Error Response

```
{
  "success" : false,
  "message" : "Validation failed",
  "errors" : ["email is required", "mobile must be 10 digits"],
  "timestamp" : "2026-06-28T10:00:00Z",
  "statusCode": 400
}
```

## Pagination Standard

```
GET /api/v1/loans?page=0&size=10&sortBy=createdAt&sortDir=desc
```

Response:

```
{
  "data": {
    "content"      : [ ],
    "page"         : 0,
    "size"         : 10,
    "totalElements": 150,
    "totalPages"   : 15,
    "last"         : false
  }
}
```

## 2. Authentication APIs

---

### Overview

Authentication APIs handle user registration, login, password management, and token operations. All secured APIs require a valid JWT Bearer Token in the Authorization header.

### Base Path: /api/v1/auth

| Method | Endpoint                     | Description                 |
|--------|------------------------------|-----------------------------|
| POST   | /api/v1/auth/register        | Register a new user         |
| POST   | /api/v1/auth/login           | Login and get JWT token     |
| POST   | /api/v1/auth/logout          | Logout and invalidate token |
| POST   | /api/v1/auth/refresh-token   | Refresh expired JWT token   |
| POST   | /api/v1/auth/forgot-password | Send password reset link    |
| POST   | /api/v1/auth/reset-password  | Reset password using token  |
| POST   | /api/v1/auth/change-password | Change current password     |
| GET    | /api/v1/auth/verify-email    | Verify email via token      |

### 2.1 Register User

|             |                       |                         |
|-------------|-----------------------|-------------------------|
| <b>POST</b> | /api/v1/auth/register | Register a new customer |
|-------------|-----------------------|-------------------------|

| Property      | Value                   |
|---------------|-------------------------|
| Method        | POST                    |
| Endpoint      | /api/v1/auth/register   |
| Description   | Register a new customer |
| Auth Required | No                      |
| Roles         | PUBLIC                  |

#### Request Body:

```
{
  "firstName"    : "Rohan",
  "lastName"     : "Patil",
  "email"        : "rohan.patil@gmail.com",
  "mobileNumber" : "9876543210",
  "password"     : "SecurePass@123",
}
```

```
{
  "confirmPassword": "SecurePass@123"
}
```

#### Response Body:

```
{
  "success" : true,
  "message" : "User registered successfully. Please verify your email.",
  "data" : { "userId": 1, "email": "rohan.patil@gmail.com" },
  "statusCode": 201
}
```

## 2.2 Login

**POST**

/api/v1/auth/login

Login and receive JWT tokens

| Property      | Value                        |
|---------------|------------------------------|
| Method        | POST                         |
| Endpoint      | /api/v1/auth/login           |
| Description   | Login and receive JWT tokens |
| Auth Required | No                           |
| Roles         | PUBLIC                       |

#### Request Body:

```
{
  "email" : "rohan.patil@gmail.com",
  "password": "SecurePass@123"
}
```

#### Response Body:

```
{
  "success": true,
  "data": {
    "accessToken" : "eyJhbGciOiJIUzI1NiIsInR...\"",
    "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2...\"",
    "tokenType" : "Bearer",
    "expiresIn" : 86400,
    "role" : "CUSTOMER"
  }
}
```

## 2.3 Forgot Password

**POST**

/api/v1/auth/forgot-  
password

Send reset link to email

| Property      | Value                        |
|---------------|------------------------------|
| Method        | POST                         |
| Endpoint      | /api/v1/auth/forgot-password |
| Description   | Send reset link to email     |
| Auth Required | No                           |
| Roles         | PUBLIC                       |

#### Request Body:

```
{ "email": "rohan.patil@gmail.com" }
```

#### Response Body:

```
{ "success": true, "message": "Password reset link sent to your email.",
  "statusCode": 200 }
```

## 2.4 Reset Password

|             |                             |                            |
|-------------|-----------------------------|----------------------------|
| <b>POST</b> | /api/v1/auth/reset-password | Reset password using token |
|-------------|-----------------------------|----------------------------|

| Property      | Value                       |
|---------------|-----------------------------|
| Method        | POST                        |
| Endpoint      | /api/v1/auth/reset-password |
| Description   | Reset password using token  |
| Auth Required | No                          |
| Roles         | PUBLIC                      |

#### Request Body:

```
{
  "token"      : "reset-token-from-email",
  "newPassword" : "NewPass@456",
  "confirmPassword": "NewPass@456"
}
```

#### Response Body:

```
{ "success": true, "message": "Password reset successfully.", "statusCode": 200 }
```

## 3. Customer APIs

---

### Overview

Customer APIs manage customer profile data including personal information, address, employment details, and identity documents.

### Base Path: `/api/v1/customers`

| Method | Endpoint                                    | Description                   | Roles                  |
|--------|---------------------------------------------|-------------------------------|------------------------|
| POST   | <code>/api/v1/customers/profile</code>      | Create customer profile       | CUSTOMER               |
| GET    | <code>/api/v1/customers/profile</code>      | Get own profile               | CUSTOMER               |
| PUT    | <code>/api/v1/customers/profile</code>      | Update full profile           | CUSTOMER               |
| PATCH  | <code>/api/v1/customers/profile</code>      | Partial update                | CUSTOMER               |
| GET    | <code>/api/v1/customers/{customerId}</code> | Get customer by ID            | LOAN_OFFICER,<br>ADMIN |
| GET    | <code>/api/v1/customers</code>              | Get all customers (paginated) | ADMIN,<br>LOAN_OFFICER |
| DELETE | <code>/api/v1/customers/{customerId}</code> | Soft delete customer          | ADMIN                  |

### 3.1 Create Customer Profile

|             |                                        |                                   |
|-------------|----------------------------------------|-----------------------------------|
| <b>POST</b> | <code>/api/v1/customers/profile</code> | Create profile after registration |
|-------------|----------------------------------------|-----------------------------------|

| Property      | Value                                  |
|---------------|----------------------------------------|
| Method        | POST                                   |
| Endpoint      | <code>/api/v1/customers/profile</code> |
| Description   | Create profile after registration      |
| Auth Required | Yes                                    |
| Roles         | CUSTOMER                               |

#### Request Body:

```
{
  "firstName"    : "Rohan",
  "lastName"     : "Patil",
  "dateOfBirth"  : "1995-06-15",
  "gender"       : "MALE",
  "fatherName"   : "Suresh Patil",
```

```

"mobileNumber" : "9876543210",
"email" : "rohan.patil@gmail.com",
"panNumber" : "ABCDE1234F",
"aadhaarNumber" : "1234 5678 9012",
"address" : {
  "street" : "123 MG Road",
  "city" : "Pune",
  "state" : "Maharashtra",
  "pincode" : "411001"
},
"employmentType" : "SALARIED",
"companyName" : "TCS",
"monthlyIncome" : 75000.00
}

```

#### Response Body:

```

{
  "success": true,
  "message": "Customer profile created successfully.",
  "data" : { "customerId": 101, "panNumber": "ABCDE1234F" },
  "statusCode": 201
}

```

## 3.2 Get Own Profile

|            |                           |                                    |
|------------|---------------------------|------------------------------------|
| <b>GET</b> | /api/v1/customers/profile | Get the logged-in customer profile |
|------------|---------------------------|------------------------------------|

| Property      | Value                              |
|---------------|------------------------------------|
| Method        | GET                                |
| Endpoint      | /api/v1/customers/profile          |
| Description   | Get the logged-in customer profile |
| Auth Required | Yes                                |
| Roles         | CUSTOMER                           |

#### Response Body:

```

{
  "success": true,
  "data": {
    "customerId" : 101,
    "firstName" : "Rohan",
    "lastName" : "Patil",
    "email" : "rohan.patil@gmail.com",
    "panNumber" : "ABCDE1234F",
    "employmentType": "SALARIED",
    "monthlyIncome" : 75000.00,
    "createdAt" : "2026-01-15T09:30:00Z"
  }
}

```



## 4. Loan APIs

### Overview

Loan APIs manage the complete lifecycle of a loan application — from submission to disbursement.

**Base Path:** `/api/v1/loans`

| Method | Endpoint                                           | Description                             | Roles                  |
|--------|----------------------------------------------------|-----------------------------------------|------------------------|
| POST   | <code>/api/v1/loans</code>                         | Submit new loan application             | CUSTOMER               |
| GET    | <code>/api/v1/loans</code>                         | Get all loans (paginated)               | ADMIN,<br>LOAN_OFFICER |
| GET    | <code>/api/v1/loans/my-loans</code>                | Get own loan applications               | CUSTOMER               |
| GET    | <code>/api/v1/loans/{loanId}</code>                | Get loan by ID                          | ALL AUTH<br>ROLES      |
| PUT    | <code>/api/v1/loans/{loanId}</code>                | Update loan application<br>(DRAFT only) | CUSTOMER               |
| PATCH  | <code>/api/v1/loans/{loanId}/submit</code>         | Submit loan for review                  | CUSTOMER               |
| PATCH  | <code>/api/v1/loans/{loanId}/cancel</code>         | Cancel loan application                 | CUSTOMER               |
| GET    | <code>/api/v1/loans/{loanId}/status-history</code> | Loan status history                     | ALL AUTH<br>ROLES      |
| GET    | <code>/api/v1/loan-types</code>                    | Get all loan types                      | PUBLIC                 |

### 4.1 Submit Loan Application

|             |                            |                               |
|-------------|----------------------------|-------------------------------|
| <b>POST</b> | <code>/api/v1/loans</code> | Submit a new loan application |
|-------------|----------------------------|-------------------------------|

| Property      | Value                         |
|---------------|-------------------------------|
| Method        | POST                          |
| Endpoint      | <code>/api/v1/loans</code>    |
| Description   | Submit a new loan application |
| Auth Required | Yes                           |
| Roles         | CUSTOMER                      |

**Request Body:**

|   |
|---|
| { |
|---|

```

"loanTypeId"      : 1,
"loanAmount"      : 500000.00,
"loanTenureMonths": 36,
"purpose"         : "Home Renovation",
"employmentType"  : "SALARIED",
"monthlyIncome"   : 75000.00,
"existingEMIs"    : 5000.00
}

```

#### Response Body:

```

{
  "success": true,
  "message": "Loan application submitted successfully.",
  "data": {
    "loanId"          : 501,
    "applicationNumber": "LOAN20260628001",
    "loanStatus"       : "DRAFT",
    "loanAmount"       : 500000.00,
    "interestRate"     : 12.5,
    "createdAt"        : "2026-06-28T10:15:00Z"
  }
}

```

## 4.2 Get Loan by ID

|            |                        |                                     |
|------------|------------------------|-------------------------------------|
| <b>GET</b> | /api/v1/loans/{loanId} | Get full details of a specific loan |
|------------|------------------------|-------------------------------------|

| Property      | Value                               |
|---------------|-------------------------------------|
| Method        | GET                                 |
| Endpoint      | /api/v1/loans/{loanId}              |
| Description   | Get full details of a specific loan |
| Auth Required | Yes                                 |
| Roles         | ALL AUTH ROLES                      |

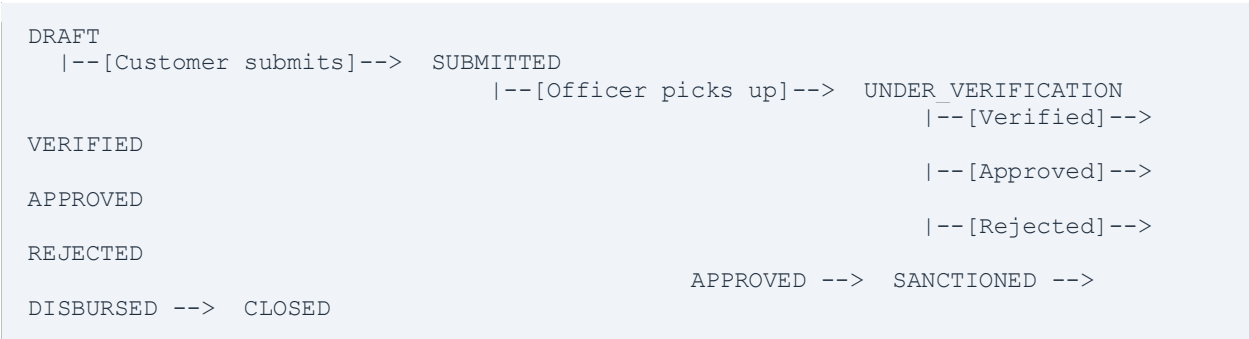
#### Response Body:

```

{
  "success": true,
  "data": {
    "loanId"          : 501,
    "applicationNumber": "LOAN20260628001",
    "loanStatus"       : "UNDER_VERIFICATION",
    "loanAmount"       : 500000.00,
    "loanTenureMonths" : 36,
    "interestRate"     : 12.5,
    "emi"              : 16680.00,
    "customer"         : { "customerId": 101, "name": "Rohan Patil" },
    "loanType"         : { "loanTypeId": 1, "name": "Personal Loan" }
  }
}

```

### 4.3 Loan Status Flow



# 5. Document APIs

## Overview

Document APIs manage uploading, retrieving, and deleting customer KYC and income documents required for loan processing.

Base Path: `/api/v1/documents`

| Method | Endpoint                                             | Description             | Roles           |
|--------|------------------------------------------------------|-------------------------|-----------------|
| POST   | <code>/api/v1/documents/upload</code>                | Upload document         | CUSTOMER        |
| GET    | <code>/api/v1/documents/loan/{loanId}</code>         | Get all docs for a loan | ALL AUTH ROLES  |
| GET    | <code>/api/v1/documents/{documentId}</code>          | Get document metadata   | ALL AUTH ROLES  |
| DELETE | <code>/api/v1/documents/{documentId}</code>          | Delete document         | CUSTOMER, ADMIN |
| GET    | <code>/api/v1/documents/{documentId}/download</code> | Download document file  | ALL AUTH ROLES  |

## 5.1 Upload Document

|      |                                       |                                                  |
|------|---------------------------------------|--------------------------------------------------|
| POST | <code>/api/v1/documents/upload</code> | Upload KYC/income document (multipart/form-data) |
|------|---------------------------------------|--------------------------------------------------|

| Property      | Value                                            |
|---------------|--------------------------------------------------|
| Method        | POST                                             |
| Endpoint      | <code>/api/v1/documents/upload</code>            |
| Description   | Upload KYC/income document (multipart/form-data) |
| Auth Required | Yes                                              |
| Roles         | CUSTOMER                                         |

### Request Body:

```
Content-Type: multipart/form-data

Form Fields:
  loanId      : 501
  documentType: PAN_CARD | AADHAAR_CARD | SALARY_SLIP | BANK_STATEMENT | PHOTO
  file        : <binary file>
```

Response Body:

```
{
  "success": true,
  "message": "Document uploaded successfully.",
  "data": {
    "documentId" : 201,
    "documentType": "PAN_CARD",
    "fileName"    : "PAN_101.pdf",
    "fileSize"    : 204800,
    "uploadedAt"  : "2026-06-28T10:20:00Z"
  }
}
```

Supported Document Types

| Document Type  | Description                 | Required                  |
|----------------|-----------------------------|---------------------------|
| PAN_CARD       | Government identity proof   | Mandatory                 |
| AADHAAR_CARD   | Address + identity proof    | Mandatory                 |
| SALARY_SLIP    | Latest 3 months salary slip | Mandatory (Salaried)      |
| BANK_STATEMENT | Last 6 months statement     | Mandatory                 |
| ITR            | Income Tax Return (2 years) | Mandatory (Self-Employed) |
| PASSPORT_PHOTO | Recent photograph           | Mandatory                 |
| FORM_16        | Tax deduction certificate   | Optional                  |

## 6. Approval APIs

### Overview

Approval APIs implement the Maker-Checker workflow. A Loan Officer (Maker) reviews and recommends. A Credit Manager (Checker) approves or rejects. This two-level process ensures accountability and prevents fraud.

### Base Path: /api/v1/approvals

| Method | Endpoint                            | Description              | Role            |
|--------|-------------------------------------|--------------------------|-----------------|
| GET    | /api/v1/approvals/pending           | Get all pending loans    | LOAN_OFFICER    |
| POST   | /api/v1/approvals/{loanId}/verify   | Mark loan as verified    | LOAN_OFFICER    |
| POST   | /api/v1/approvals/{loanId}/approve  | Approve loan             | CREDIT_MANAGER  |
| POST   | /api/v1/approvals/{loanId}/reject   | Reject loan              | CREDIT_MANAGER  |
| POST   | /api/v1/approvals/{loanId}/sanction | Sanction approved loan   | FINANCE_OFFICER |
| POST   | /api/v1/approvals/{loanId}/disburse | Disburse sanctioned loan | FINANCE_OFFICER |
| GET    | /api/v1/approvals/{loanId}/history  | Full approval history    | ALL AUTH ROLES  |

### 6.1 Verify Loan (Maker)

|             |                                   |                                |
|-------------|-----------------------------------|--------------------------------|
| <b>POST</b> | /api/v1/approvals/{loanId}/verify | Loan Officer verifies the loan |
|-------------|-----------------------------------|--------------------------------|

| Property      | Value                             |
|---------------|-----------------------------------|
| Method        | POST                              |
| Endpoint      | /api/v1/approvals/{loanId}/verify |
| Description   | Loan Officer verifies the loan    |
| Auth Required | Yes                               |
| Roles         | LOAN_OFFICER                      |

#### Request Body:

|                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------|
| <pre>{   "remarks": "All documents verified. Income eligibility confirmed.",   "recommendedAmount": 500000.00 }</pre> |
|-----------------------------------------------------------------------------------------------------------------------|

#### Response Body:

```
{ "success": true, "message": "Loan marked as VERIFIED.", "data": { "loanStatus": "VERIFIED" }, "statusCode": 200 }
```

## 6.2 Approve Loan (Checker)

|             |                                    |                                  |
|-------------|------------------------------------|----------------------------------|
| <b>POST</b> | /api/v1/approvals/{loanId}/approve | Credit Manager approves the loan |
|-------------|------------------------------------|----------------------------------|

| Property      | Value                              |
|---------------|------------------------------------|
| Method        | POST                               |
| Endpoint      | /api/v1/approvals/{loanId}/approve |
| Description   | Credit Manager approves the loan   |
| Auth Required | Yes                                |
| Roles         | CREDIT_MANAGER                     |

### Request Body:

```
{  "approvedAmount": 500000.00,  "interestRate"   : 12.5,  "remarks"        : "Approved based on CIBIL score and income."}
```

### Response Body:

```
{ "success": true, "message": "Loan APPROVED.", "data": { "loanStatus": "APPROVED", "approvedAmount": 500000.00 }, "statusCode": 200 }
```

## 6.3 Reject Loan

|             |                                   |                                 |
|-------------|-----------------------------------|---------------------------------|
| <b>POST</b> | /api/v1/approvals/{loanId}/reject | Credit Manager rejects the loan |
|-------------|-----------------------------------|---------------------------------|

| Property      | Value                             |
|---------------|-----------------------------------|
| Method        | POST                              |
| Endpoint      | /api/v1/approvals/{loanId}/reject |
| Description   | Credit Manager rejects the loan   |
| Auth Required | Yes                               |
| Roles         | CREDIT_MANAGER                    |

### Request Body:

```
{  "rejectionReason": "Low CIBIL score. Score below 650.",}
```

```
    "remarks"      : "Customer may re-apply after 6 months."
  }
```

### Response Body:

```
{ "success": true, "message": "Loan REJECTED.", "data": { "loanStatus": "REJECTED"
}, "statusCode": 200 }
```

# 7. EMI APIs

## Overview

EMI APIs handle EMI calculation, schedule generation, and repayment tracking for approved loans.

Base Path: `/api/v1/emi`

| Method | Endpoint                                   | Description                     | Roles                  |
|--------|--------------------------------------------|---------------------------------|------------------------|
| POST   | <code>/api/v1/emi/calculate</code>         | Calculate EMI (before applying) | PUBLIC                 |
| GET    | <code>/api/v1/emi/{loanId}/schedule</code> | Get full EMI schedule           | CUSTOMER, LOAN_OFFICER |
| GET    | <code>/api/v1/emi/{loanId}/summary</code>  | Get EMI summary                 | CUSTOMER, LOAN_OFFICER |

## 7.1 Calculate EMI

|      |                                    |                                         |
|------|------------------------------------|-----------------------------------------|
| POST | <code>/api/v1/emi/calculate</code> | Calculate EMI without applying for loan |
|------|------------------------------------|-----------------------------------------|

| Property      | Value                                   |
|---------------|-----------------------------------------|
| Method        | POST                                    |
| Endpoint      | <code>/api/v1/emi/calculate</code>      |
| Description   | Calculate EMI without applying for loan |
| Auth Required | No                                      |
| Roles         | PUBLIC                                  |

### Request Body:

```
{
  "loanAmount"      : 500000,
  "interestRate"     : 12.5,
  "loanTenureMonths": 36
}
```

### Response Body:

```
{
  "success": true,
  "data": {
    "loanAmount"      : 500000.00,
    "interestRate"     : 12.5,
  }
}
```

```
"tenureMonths"    : 36,  
"monthlyEMI"      : 16680.00,  
"totalInterest"   : 100480.00,  
"totalRepayment"  : 600480.00  
}  
}
```

## EMI Formula

$$\text{EMI} = P \times r \times (1+r)^n / ((1+r)^n - 1)$$

Where:

P = Principal Loan Amount

r = Monthly Interest Rate (Annual Rate / 12 / 100)

n = Loan Tenure in Months

Example:

P = 500000, Annual Rate = 12.5%, n = 36

r = 12.5 / 12 / 100 = 0.010417

EMI =  $500000 \times 0.010417 \times (1.010417)^{36} / ((1.010417)^{36} - 1)$

EMI = ₹16,680 (approx)

## 8. Notification APIs

---

### Overview

Notification APIs manage sending and retrieving notifications. Notifications are triggered automatically at key loan lifecycle events.

**Base Path:** `/api/v1/notifications`

| Method | Endpoint                                     | Description                            | Roles    |
|--------|----------------------------------------------|----------------------------------------|----------|
| GET    | <code>/api/v1/notifications</code>           | Get all notifications (logged-in user) | ALL AUTH |
| GET    | <code>/api/v1/notifications/unread</code>    | Get unread count                       | ALL AUTH |
| PATCH  | <code>/api/v1/notifications/{id}/read</code> | Mark as read                           | ALL AUTH |
| PATCH  | <code>/api/v1/notifications/read-all</code>  | Mark all as read                       | ALL AUTH |
| DELETE | <code>/api/v1/notifications/{id}</code>      | Delete notification                    | ALL AUTH |

### Auto-Trigged Notifications

| Event                   | Notification Type    | Channel            |
|-------------------------|----------------------|--------------------|
| User Registration       | Welcome Email        | EMAIL              |
| Loan Submitted          | Application Received | EMAIL, SMS         |
| Documents Required      | Documents Pending    | EMAIL, IN-APP      |
| Loan Under Verification | Under Review         | EMAIL, IN-APP      |
| Loan Approved           | Congratulations!     | EMAIL, SMS, IN-APP |
| Loan Rejected           | Application Update   | EMAIL, SMS         |
| Loan Disbursed          | Amount Credited      | EMAIL, SMS, IN-APP |
| EMI Reminder            | EMI Due Alert        | EMAIL, SMS, IN-APP |

# 9. Admin APIs

## Overview

Admin APIs are accessible only to users with the ADMIN role. They provide system management, user management, and reporting capabilities.

Base Path: `/api/v1/admin`

| Method | Endpoint                                         | Description                                  |
|--------|--------------------------------------------------|----------------------------------------------|
| GET    | <code>/api/v1/admin/users</code>                 | Get all users (paginated)                    |
| GET    | <code>/api/v1/admin/users/{userId}</code>        | Get user by ID                               |
| PATCH  | <code>/api/v1/admin/users/{userId}/status</code> | Change user status (ACTIVE/LOCKED/SUSPENDED) |
| DELETE | <code>/api/v1/admin/users/{userId}</code>        | Soft delete user                             |
| GET    | <code>/api/v1/admin/loans</code>                 | Get all loan applications                    |
| GET    | <code>/api/v1/admin/reports/loans</code>         | Loan summary report                          |
| GET    | <code>/api/v1/admin/reports/customers</code>     | Customer summary report                      |
| GET    | <code>/api/v1/admin/audit-logs</code>            | Get audit logs                               |
| POST   | <code>/api/v1/admin/loan-types</code>            | Create new loan type                         |
| PUT    | <code>/api/v1/admin/loan-types/{id}</code>       | Update loan type                             |
| DELETE | <code>/api/v1/admin/loan-types/{id}</code>       | Delete loan type                             |

## 9.1 Change User Status

|       |                                                  |                                  |
|-------|--------------------------------------------------|----------------------------------|
| PATCH | <code>/api/v1/admin/users/{userId}/status</code> | Activate, lock or suspend a user |
|-------|--------------------------------------------------|----------------------------------|

| Property      | Value                                            |
|---------------|--------------------------------------------------|
| Method        | PATCH                                            |
| Endpoint      | <code>/api/v1/admin/users/{userId}/status</code> |
| Description   | Activate, lock or suspend a user                 |
| Auth Required | Yes                                              |
| Roles         | ADMIN                                            |

Request Body:

```
{  
  "status" : "LOCKED",  
  "reason" : "Multiple failed login attempts"  
}
```

#### Response Body:

```
{ "success": true, "message": "User status updated to LOCKED.", "statusCode": 200  
}
```

# 10. Error Responses

## Standard Error Format

```
{
  "success"    : false,
  "message"    : "Descriptive error message",
  "errors"     : [ "field-level detail 1", "field-level detail 2" ],
  "timestamp"  : "2026-06-28T10:00:00Z",
  "statusCode" : 400
}
```

## Common Error Scenarios

| Scenario                | HTTP Code | Error Message                                         |
|-------------------------|-----------|-------------------------------------------------------|
| Validation Failed       | 400       | email is required / mobile must be 10 digits          |
| Unauthorized (No token) | 401       | Authentication token is missing or invalid            |
| Forbidden (Wrong role)  | 403       | You do not have permission to access this resource    |
| Resource Not Found      | 404       | Loan application with ID 501 not found                |
| Duplicate Email         | 409       | Email address is already registered                   |
| Duplicate PAN           | 409       | PAN number already exists in the system               |
| Validation Error        | 422       | Loan amount must be between 10,000 and 5,000,000      |
| Internal Server Error   | 500       | An unexpected error occurred. Please try again later. |
| Service Unavailable     | 503       | Service is temporarily unavailable.                   |

## Validation Error Example

```
{
  "success"    : false,
  "message"    : "Validation failed",
  "errors"     : [
    "firstName is required",
    "email must be a valid email address",
    "mobileNumber must be exactly 10 digits",
    "panNumber format is invalid"
  ],
  "statusCode" : 400
}
```

## 11. Status Codes

---

### HTTP Status Codes Used

| Code | Name                  | When Used                                    |
|------|-----------------------|----------------------------------------------|
| 200  | OK                    | Successful GET, PUT, PATCH                   |
| 201  | Created               | Successful POST (resource created)           |
| 204  | No Content            | Successful DELETE                            |
| 400  | Bad Request           | Invalid input / validation failure           |
| 401  | Unauthorized          | Missing or invalid JWT token                 |
| 403  | Forbidden             | Valid token but insufficient role/permission |
| 404  | Not Found             | Requested resource does not exist            |
| 409  | Conflict              | Duplicate email, PAN, or mobile number       |
| 422  | Unprocessable Entity  | Business rule violation                      |
| 429  | Too Many Requests     | Rate limit exceeded                          |
| 500  | Internal Server Error | Unexpected server-side error                 |
| 503  | Service Unavailable   | Maintenance or downstream failure            |

# 12. API Security

## Security Mechanisms

| Mechanism                | Details                                                        |
|--------------------------|----------------------------------------------------------------|
| JWT Authentication       | Bearer token in Authorization header — expires in 24 hours     |
| Refresh Token            | Long-lived token to get new access tokens without re-login     |
| Role-Based Access        | CUSTOMER, LOAN_OFFICER, CREDIT_MANAGER, FINANCE_OFFICER, ADMIN |
| HTTPS Only               | All APIs only served over TLS/SSL                              |
| Password Hashing         | BCrypt with strength 10 — never stored in plaintext            |
| Input Validation         | All inputs validated at controller layer (Bean Validation)     |
| SQL Injection Prevention | Spring Data JPA + Parameterized Queries                        |
| Rate Limiting            | Max 100 requests/minute per IP — returns 429 if exceeded       |
| CORS                     | Only whitelisted origins allowed                               |
| Audit Logging            | All API calls logged with user, timestamp, and action          |

## JWT Token Structure

```
Header:
{ "alg": "HS256", "typ": "JWT" }

Payload:
{
  "sub"      : "rohan.patil@gmail.com",
  "userId"   : 1,
  "role"     : "CUSTOMER",
  "iat"      : 1751100000,
  "exp"      : 1751186400
}

Authorization Header Usage:
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

## Role Permissions Matrix

| Action           | CUSTOMER | LOAN_OFFICER | CREDIT_MANAGER | FINANCE_OFFICER | ADMIN |
|------------------|----------|--------------|----------------|-----------------|-------|
| Register / Login | YES      | YES          | YES            | YES             | YES   |
| View Own Profile | YES      | NO           | NO             | NO              | YES   |

| Action              | CUSTOMER | LOAN_OFFICER | CREDIT_MANAGER | FINANCE_OFFICER | ADMIN |
|---------------------|----------|--------------|----------------|-----------------|-------|
| Submit Loan         | YES      | NO           | NO             | NO              | NO    |
| Upload Documents    | YES      | NO           | NO             | NO              | NO    |
| View All Loans      | NO       | YES          | YES            | YES             | YES   |
| Verify Loan         | NO       | YES          | NO             | NO              | NO    |
| Approve / Reject    | NO       | NO           | YES            | NO              | NO    |
| Sanction / Disburse | NO       | NO           | NO             | YES             | NO    |
| Manage Users        | NO       | NO           | NO             | NO              | YES   |
| View Audit Logs     | NO       | NO           | NO             | NO              | YES   |

# 13. API Versioning

## Versioning Strategy

URL path versioning is used. The version number is embedded in the API base URL.

```
Current Version:  /api/v1/
Future Version :  /api/v2/

Examples:
GET  /api/v1/loans
POST /api/v1/auth/login
GET  /api/v2/loans      (future - backward compatible)
```

## Versioning Rules

- v1 will remain active for at least 12 months after v2 is released.
- Breaking changes (renamed fields, removed endpoints) always create a new version.
- Non-breaking additions (new optional fields, new endpoints) may be added to existing version.
- Deprecation notice is sent 3 months before a version is retired.

## Complete API Summary

| Module         | Base Path             | Total Endpoints |
|----------------|-----------------------|-----------------|
| Authentication | /api/v1/auth          | 8               |
| Customer       | /api/v1/customers     | 7               |
| Loan           | /api/v1/loans         | 9               |
| Document       | /api/v1/documents     | 5               |
| Approval       | /api/v1/approvals     | 7               |
| EMI            | /api/v1/emi           | 3               |
| Notification   | /api/v1/notifications | 5               |
| Admin          | /api/v1/admin         | 11              |

Total API Endpoints: 55+

## Future API Enhancements

- GraphQL API support for flexible querying.
- WebSocket support for real-time loan status updates.
- Co-Applicant and Guarantor APIs.

- Credit Score Integration API (CIBIL / Experian).
- Payment Gateway APIs for EMI collection.
- Loan Foreclosure and Pre-Closure APIs.
- Branch Management APIs.
- Multi-Tenant Banking APIs.